

Lecture 23 Notes : Advanced Matplotlib & Data from the Web

1. Data Formats & Reading from URLs

In real projects, data often comes from **remote sources** instead of local files.

Common formats:

- **CSV** (Comma-Separated Values)
 - Text file: one row per line, values separated by commas
 - Read with: `pandas.read_csv(url_or_path)`
- **JSON** (JavaScript Object Notation)
 - Hierarchical / nested structure
 - Read with: `pandas.read_json(url_or_path)`

Key idea: Pandas can read **directly from a URL**, so you don't have to manually download files.

```
import pandas as pd

csv_url = "https://example.com/data.csv"
df = pd.read_csv(csv_url)      # For CSV

json_url = "https://example.com/data.json"
df_json = pd.read_json(json_url) # For JSON
```

2. Standard Imports & Style

For advanced plotting, you typically need:

```
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Consistent, clean look
```

```
plt.style.use('seaborn-v0_8-whitegrid')
```

- `numpy` → numerical arrays, math
- `pandas` → tabular data, time series
- `matplotlib.pyplot` → plotting
- A global style (like `seaborn-whitegrid`) makes all plots look nicer by default.

3. Fetching & Plotting Online Weather Data

Example: **Seattle weather** from the Vega datasets repo (online CSV).

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
url = 'https://raw.githubusercontent.com/vega/vega-datasets/main/data/seattle-weather.csv'
```

```
# Read CSV and parse 'date' as a datetime column
```

```
df_weather = pd.read_csv(url, parse_dates=['date'])
```

```
# Use the date as the index (good for time-series plots)
```

```
df_weather.set_index('date', inplace=True)
```

```
# Plot daily maximum temperature
```

```
plt.figure(figsize=(12, 6))
```

```
df_weather['temp_max'].plot(color='crimson')
```

```
plt.title('Maximum Daily Temperature in Seattle', fontsize=16)
```

```
plt.ylabel('Temperature (°C)')
```

```
plt.xlabel('Date')
```

```
plt.show()
```

Concepts:

- `parse_dates=['date']` → converts strings to `datetime`.
- `set_index('date')` → makes time-based plots easier.
- Pandas Series `.plot()` integrates nicely with Matplotlib.

4. Multi-Plot Layouts with `plt.subplots`

For multiple related plots in **one figure**, use subplots.

```
fig, axes = plt.subplots(2, 1, figsize=(12, 8), sharex=True)
```

- `fig` → the overall figure (canvas)
- `axes` → array of Axes objects (individual plots)
- `2, 1` → 2 rows, 1 column
- `sharex=True` → share the same x-axis (dates align across plots)

Example: Max temperature and precipitation

```
fig, axes = plt.subplots(2, 1, figsize=(12, 8), sharex=True)
```

```
# First subplot: Maximum temperature
```

```
axes[0].plot(df_weather.index, df_weather['temp_max'], color='crimson')
```

```
axes[0].set_title('Maximum Temperature Trend')
```

```
axes[0].set_ylabel('Temp (°C)')
```

```
# Second subplot: Daily precipitation
```

```
axes[1].plot(df_weather.index, df_weather['precipitation'], color='royalblue')
```

```
axes[1].set_title('Daily Precipitation')
```

```
axes[1].set_ylabel('Precipitation (mm)')
```

```
# Overall figure title and layout
```

```
fig.suptitle('Seattle Weather Analysis', fontsize=16)
```

```
plt.tight_layout(rect=[0, 0, 1, 0.96]) # leave space for suptitle
plt.show()
```

Concepts:

- `axes[0]` , `axes[1]` → select specific subplot
- `fig.suptitle()` → title for the **whole figure**
- `plt.tight_layout()` → fixes overlapping labels and titles

Another common pattern (min temperature + wind speed):

```
fig, axes = plt.subplots(2, 1, figsize=(12, 8), sharex=True)

axes[0].plot(df_weather.index, df_weather['temp_min'], color='deepskyblue')
axes[0].set_title('Minimum Daily Temperature')
axes[0].set_ylabel('Temp (°C)')

axes[1].plot(df_weather.index, df_weather['wind'], color='slategray')
axes[1].set_title('Average Daily Wind Speed')
axes[1].set_ylabel('Speed (m/s)')

fig.suptitle('Seattle Weather Metrics', fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

5. Curve Fitting & Extrapolation with NumPy

To visualize **future trends**, you can fit a curve to existing data and extend it.

Key functions:

- `np.polyfit(x, y, deg)` → returns polynomial coefficients (degree `deg`)
- `np.poly1d(coeffs)` → turns coefficients into a callable polynomial function